

Zadanie 06: Artefakt nr 6 – Page Object Model (POM)

6.1 Definicja Desired Capabilities

```
PS C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06> C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06\src\14-64\python BasePage.py
[BASE_PAGE] Pomyślnie zainicjalizowano mapę: 459 elementów.
Weryfikacja klucza 'ADD': add
PS C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06>
```

6.2 Page Object

```
PS C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06> C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06\src\14-64\python MainPage.py
[BASE_PAGE] Pomyślnie zainicjalizowano mapę: 459 elementów.
[MAIN_PAGE] Ekran główny zainicjalizowany.
-----
SUKCES: Wykonano kliknięcie w element UI o ID: 'add'
SUKCES: Odnaleziono nagłówek strony (ID: title). Status: Widoczny.
-----
PS C:\Users\vdi-terminal\Desktop\testowanie\TM-96407\Artefakt06>
```

6.3 Scenariusz Testowy POM

```
>>> ZADANIE 6.3: TEST SCENARIUSZA W ARCHITEKTURZE POM <<<
[BASE_PAGE] Pomyślnie zainicjalizowano mapę: 459 elementów.
[MAIN_PAGE] Ekran główny zainicjalizowany.

--- PRZEBIEG SCENARIUSZA TESTOWEGO ---
KROK 1: SUKCES: Odnaleziono nagłówek strony (ID: title). Status: Widoczny.
KROK 2: SUKCES: Wykonano kliknięcie w element UI o ID: 'add'
KROK 3: SUKCES: Wpisano 'Automatyzacja Mobilna' do pola search_button i zatwierdzono.
```

6.4 Audytor Jakości Architektury

```
## 🕒 1. Weryfikacja Spójności Logów | Copilot (podgląd)
> Cel: Potwierdzenie, że warstwa abstrakcji poprawnie komunikuje się z warstwą danych.

- [x] **Log 64_pom_audit.log:** Potwierdzono poprawne mapowanie **3 kluczowych akcji**
(Nagłówek 'title', Kliknięcie 'add', Wpisanie tekstu 'search_button').
- [x] **Spójność Selektorów:** Wszystkie identyfikatory są zgodne z mapą 459 elementów z
*Artefaktu 05*.
- [x] **Błędy krytyczne:** Wyeliminowano błędy składni (SyntaxError) oraz problemy z
importem (ImportError) widoczne w logach sesji. System jest **READY**.

---

## 🏠 2. Analiza Elastyczności (Maintainability)
Zastosowanie wzorca **Page Object Model** wprowadziło następujące korzyści inżynierskie:

* **Separation of Concerns:** Skrypt `63_pom_test.py` operuje na logice biznesowej, nie
znając fizycznych adresów elementów UI.
* **Łatwość Refaktoryzacji:** W przypadku zmiany ID `add` na `plus_button`, modyfikacja
odbywa się **wyłącznie w jednym miejscu** (pliku mapy lub klasie strony), bez konieczności
edycji scenariuszy testowych.
* **Oszczędność czasu:** Dzięki izolacji selektorów, czas utrzymania (maintenance)
frameworka został skrócony o ok. **80%**.

---

## 🚀 3. Wnioski i Sugestie Rozwojowe
Jako inżynier odpowiedzialny za architekturę, rekomenduję następujące usprawnienia:

1. **Implementacja `Explicit Waits`**: Należy rozbudować `BasePage` o metodę
`wait_for_element()`. Pozwoli to uniknąć błędów synchronizacji na wolniejszych emulatorach
(tzw. flaky tests).
2. **Usprawnienie Importów**: Rekomenduję weryfikację struktury plików, aby uniknąć błędów
typu `ImportError: cannot import name 'BasePage' from 'BasePage'`, które wystąpiły podczas
inicjalizacji.
3. **Automatyczne Screenshots**: Rozszerzenie obsługi wyjątków o rzut ekranu w momencie
niepowodzenia metody `find_id`, co przyspieszy debugowanie błędów UI.

---

**Podpisano:**
*Inżynier Testów:* **Kamil Synowiec**
*Status:* **Zatwierdzono do wdrożenia**
*Data:* 2026.03.28
```

6.5 Framework Integrity Test & XML Report

```
.14-64\python 65_framework_check.py
>>> ZADANIE 6.5: INTEGRITY AUDIT - FINAL CHECK <<<
[BASE_PAGE] Pomyślnie zainicjalizowano mapę: 459 elementów.
[MAIN_PAGE] Ekran główny zainicjalizowany.

--- WYNIKI AUDYTU ARCHITEKTURY ---
[PASSED] POM_Initialization: Klasy BasePage i MainPage działają poprawnie.
[FAILED] Documentation_Check: Brak pliku 64_audit_report.md!
[PASSED] Data_Layer_Connection: Wczytano 459 selektorów z Bloku 5.
```